LTRDisplay Control - End-to-End Implementation and Usage Guide

1. Executive Summary

LTRDisplay is a Power Apps Component Framework (PCF) control for model-driven apps that helps users browse Long Term Retention (LTR) data in a familiar grid-and-form experience.

The control is designed for archive-first usage in Dataverse:

- Fetch retained records with a selected view clause
- Replay cached data without refetching
- Open row details inside the same control
- Review audit changes and related records
- Minimize retained query calls through user-local caching and lazy loading

This document provides:

- Purpose and business value
- Solution design and architecture
- Import and validation steps
- Full user manual with screenshots
- Repository fork and customization workflow

2. Purpose of the Control

2.1 Problem Statement

Retention data is valuable for investigation, support, compliance, and historical analysis. However, users often need:

- Fast browsing of retained records
- Predictable filtering and navigation
- Minimal load on retained query infrastructure
- A form-like experience for details, audit, and related data

2.2 LTRDisplay Objectives

LTRDisplay addresses these needs by:

- Surfacing retained records directly in a model-driven form
- Reusing Dataverse views/forms metadata for familiarity
- Introducing cache-first interaction patterns
- Supporting drill-down across related records in one panel

3. Solution Design

3.1 Runtime Design

Main runtime behavior:

- Archive-focused mode by default
- Selected view drives retained fetch clause
- Grid renders from cached projection
- Related records load only on explicit user action
- Detail form and tabs render from metadata and selected row payload

3.2 Core Components

- PCF shell: LTRDisplayControl/index.ts
- App state orchestration: LTRDisplayControl/components/App.tsx
- Grid and local filtering: LTRDisplayControl/components/DynamicGrid.tsx
- Metadata-driven detail form: LTRDisplayControl/components/DynamicForm.tsx
- Dataverse access layer: LTRDisplayControl/services/LtrService.ts
- View/Form XML parsing: LTRDisplayControl/utils/XmlParser.ts

3.3 Cache Model

Per-user browser cache stores:

- View datasets
- Entity record dictionary by record id
- Related datasets
- Forms metadata
- Relationship metadata

This enables:

- Show Cached behavior without server refetch

- Faster row-open and navigation experience
- Reduced retained query consumption

3.4 UX and Interaction Model

- Fetch Archive button: calls retained fetch and updates cache
- Show Cached button: reads cache and applies local filtering
- Column filter flyouts: local filtering against projected rows
- Detail tabs: Summary, Record Data, Audit History, Related
- Form switcher: choose available main forms for selected entity
- Chrome toggle arrows: hide/show header and command bar behavior

3.5 Security Intent

- LTRDisplay Main Form is intended for System Administrator users
- Role-based form visibility should restrict exposure to non-admin users

4. Solution Packaging and Import

4.1 Distributed Artifacts

Latest packaged solution files:

- solution/LTRDisplay_managed_latest.zip
- solution/LTRDisplay_unmanaged_latest.zip

Unpacked inspection artifacts:

- exports/unpacked_managed
- exports/unpacked_unmanaged

4.2 Import in Power Platform (recommended managed path)

1. Open target environment in Maker portal.
2. Go to Solutions.
3. Select Import solution.
4. Upload solution/LTRDisplay_managed_latest.zip.
5. Complete import and publish customizations.

4.3 Post-Import Validation Checklist

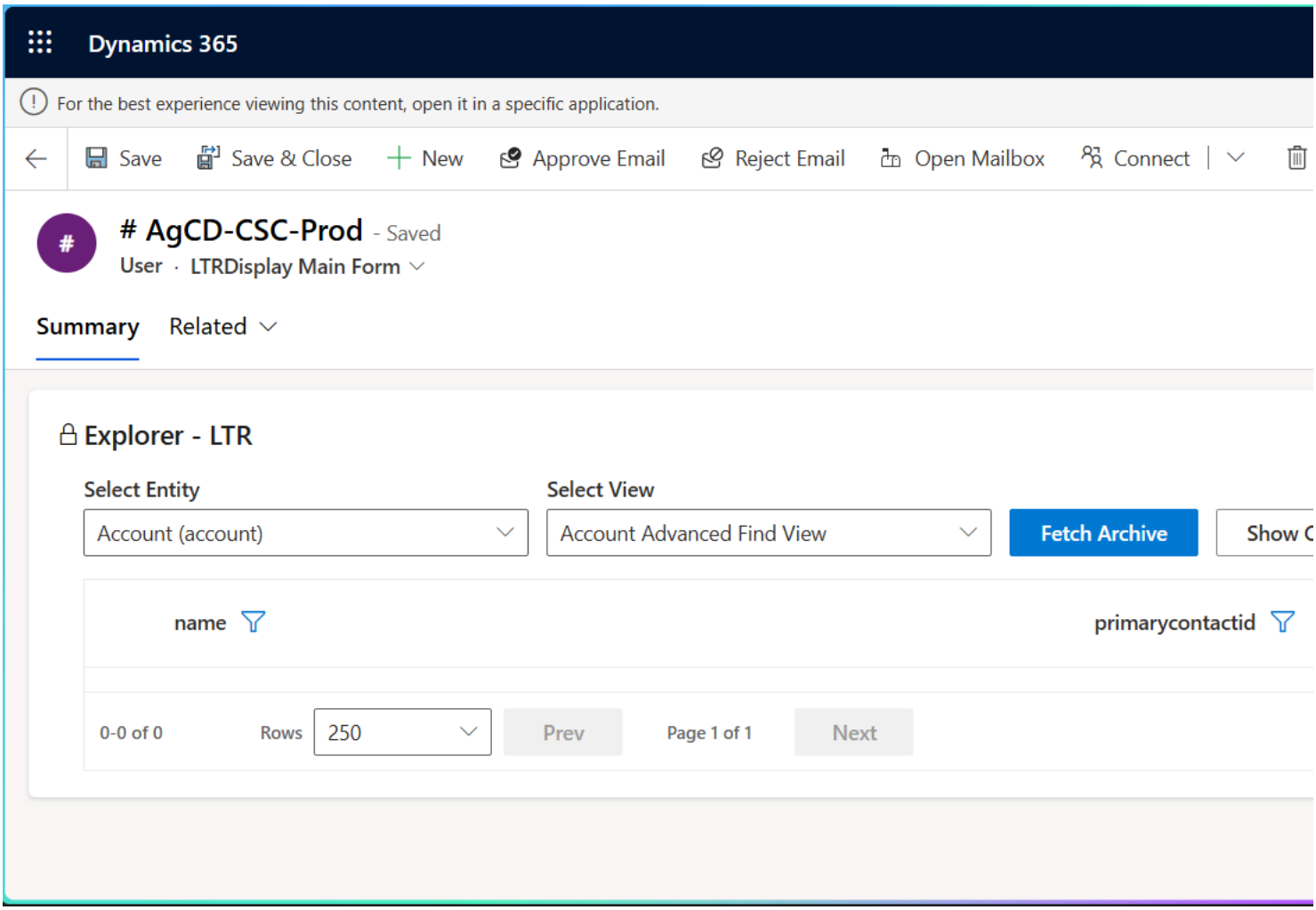
Validate the following:

- LTRDisplay Main Form exists and is enabled
- SystemUser form maps to ltr_LTRDisplay.LTRDisplayControl
- System Administrator can open the form
- Non-admin users do not get the admin-targeted form
- Fetch Archive returns retained rows
- Show Cached replays cached rows
- Record Data, Audit History, and Related tabs operate as expected

5. User Manual (Image-Based Walkthrough)

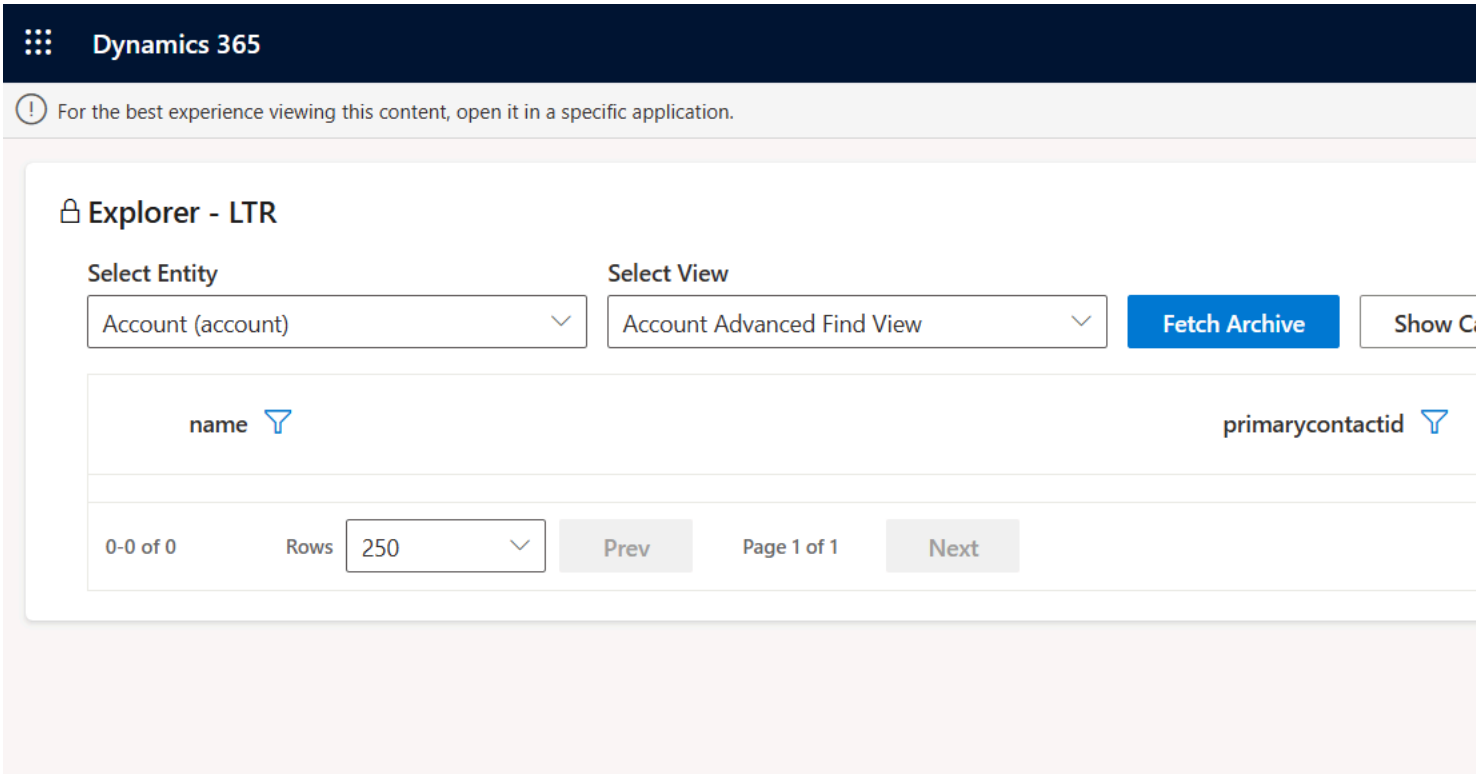
Step 1 - Open form with control visible

The form opens with Explorer - LTR and action controls.



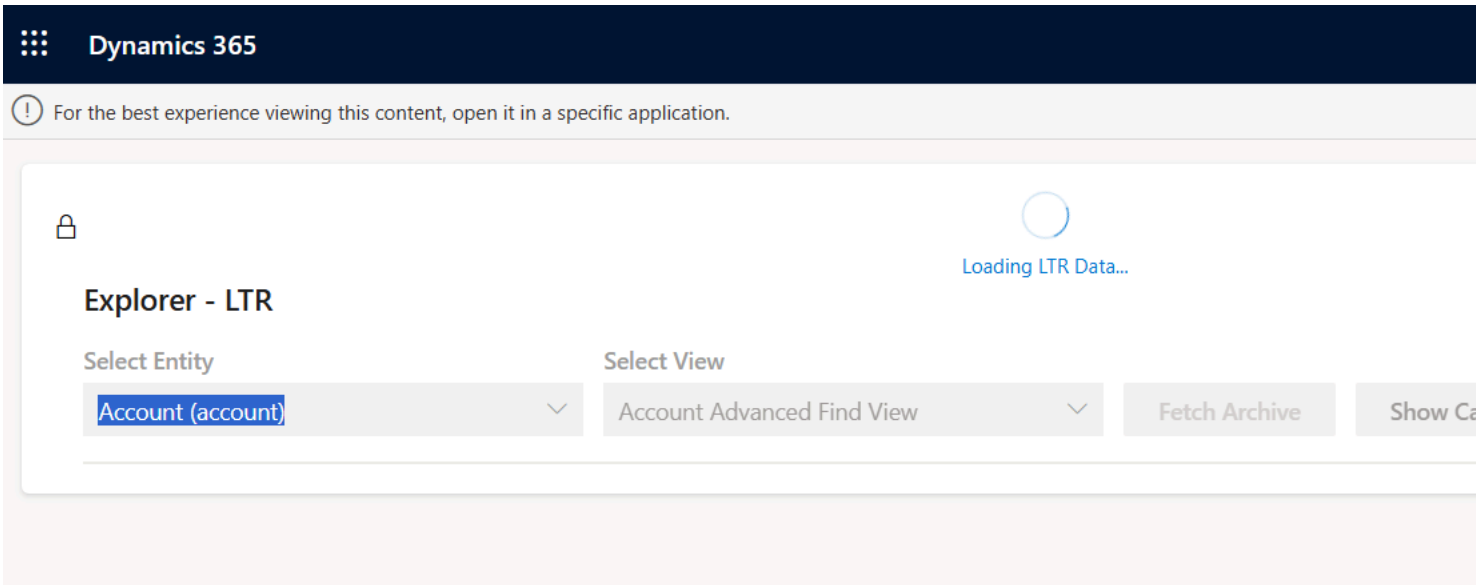
Step 2 - Toggle form chrome for focus

Use the arrow controls to hide/show header and command bar.



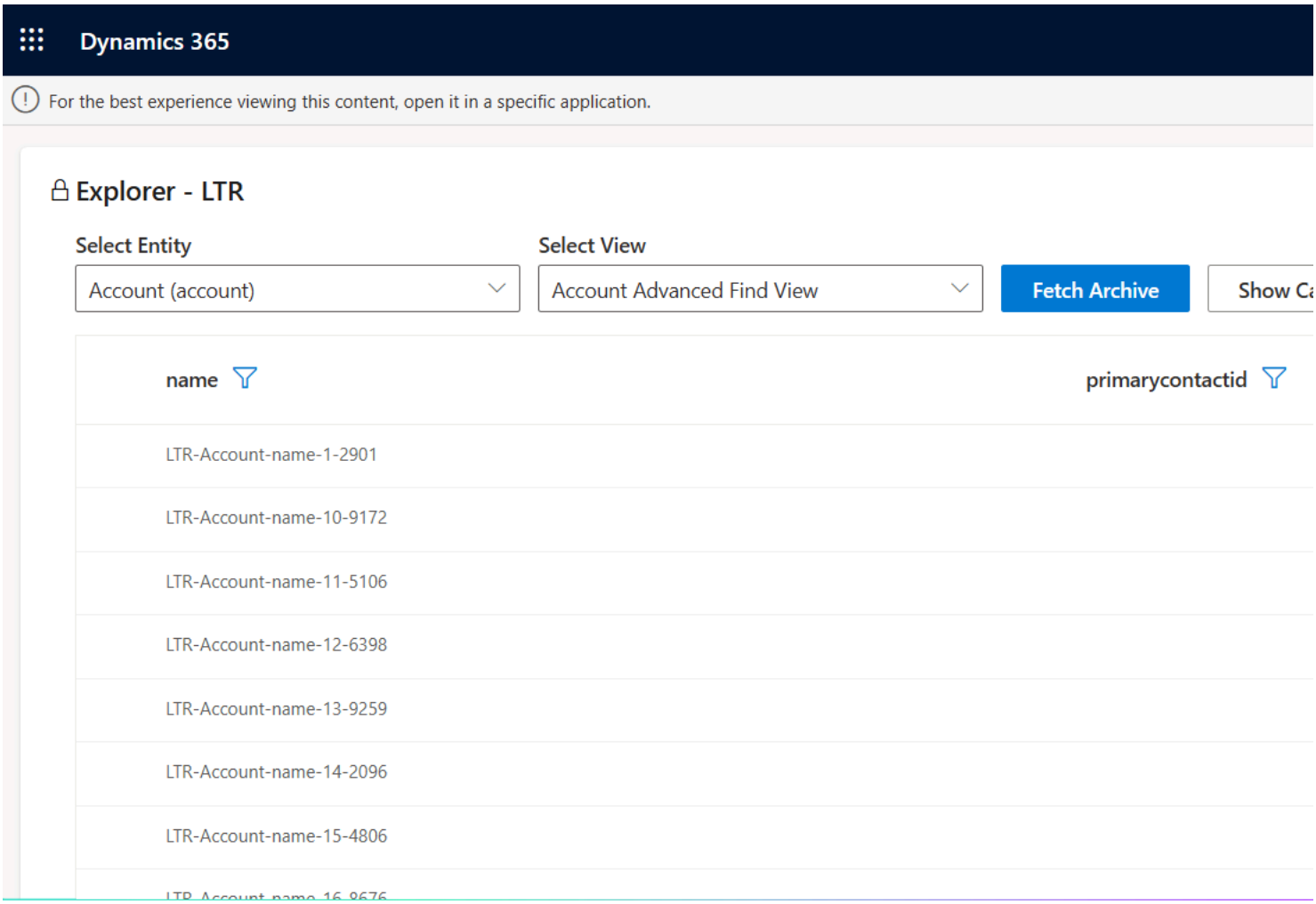
Step 3 - Start retained fetch

Click Fetch Archive. During loading, controls can be temporarily disabled.



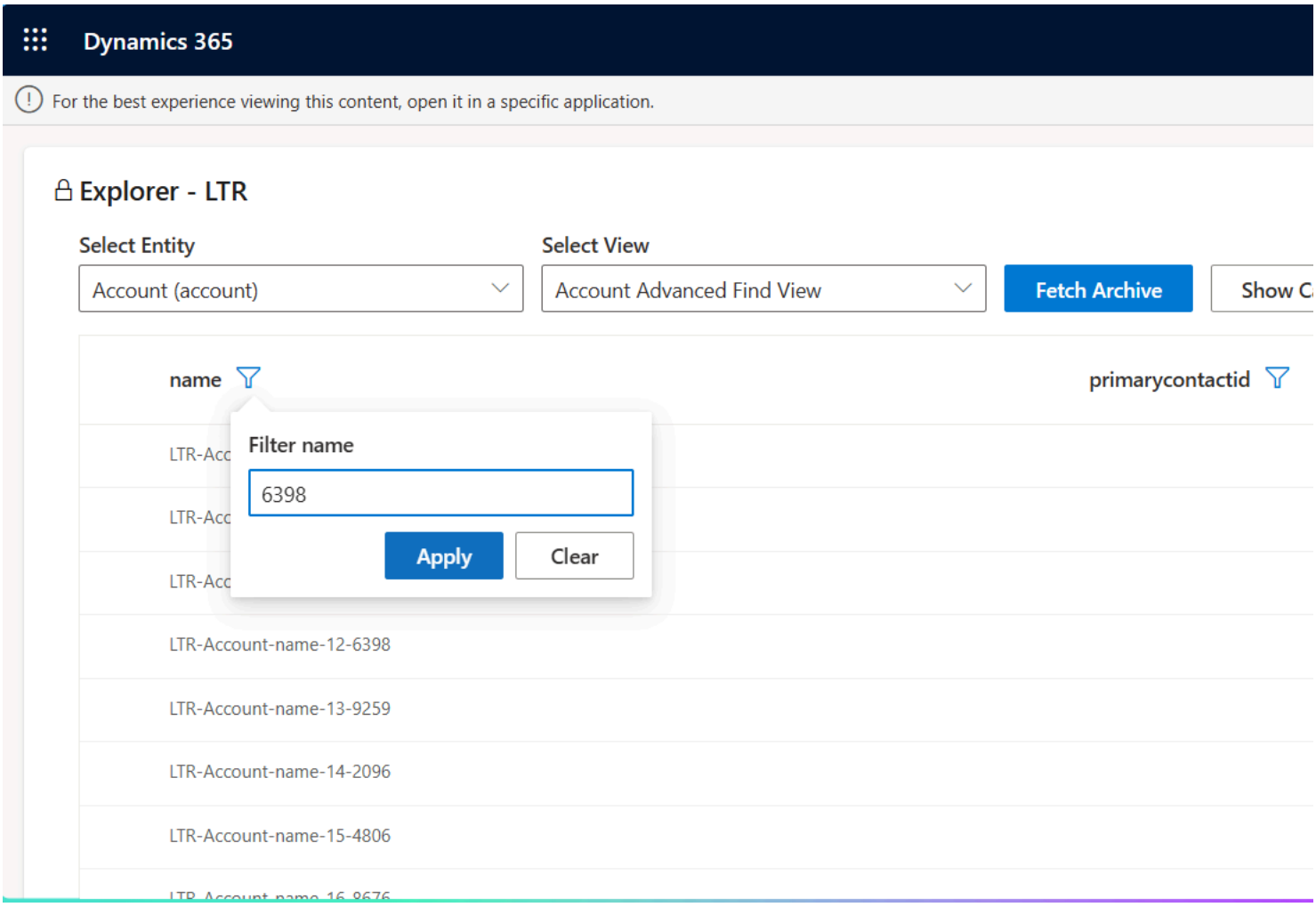
Step 4 - Review fetched grid data

Rows appear in the grid after retained fetch completes.



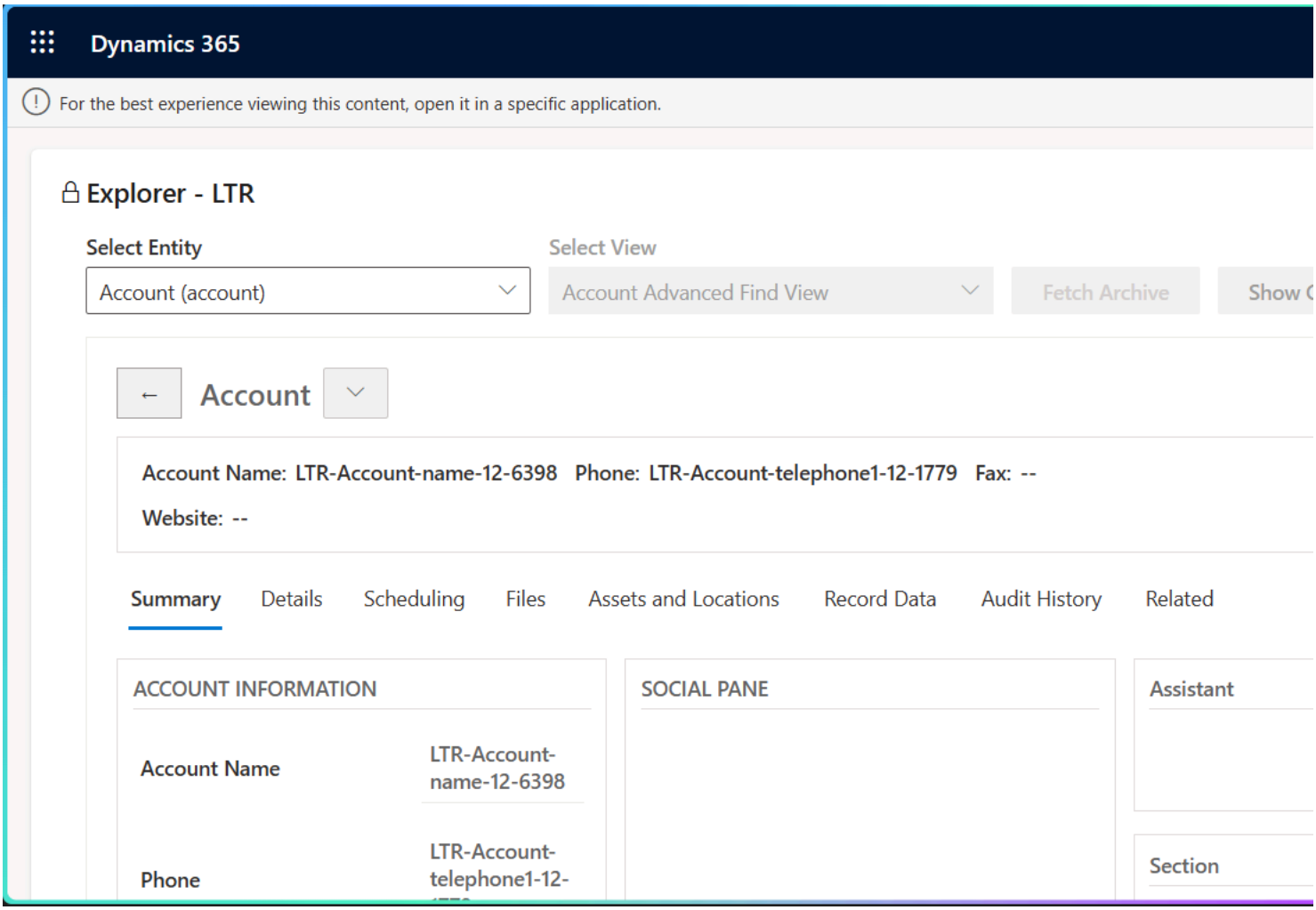
Step 5 - Apply local column filter

Open a column filter, enter a value, and apply the filter.



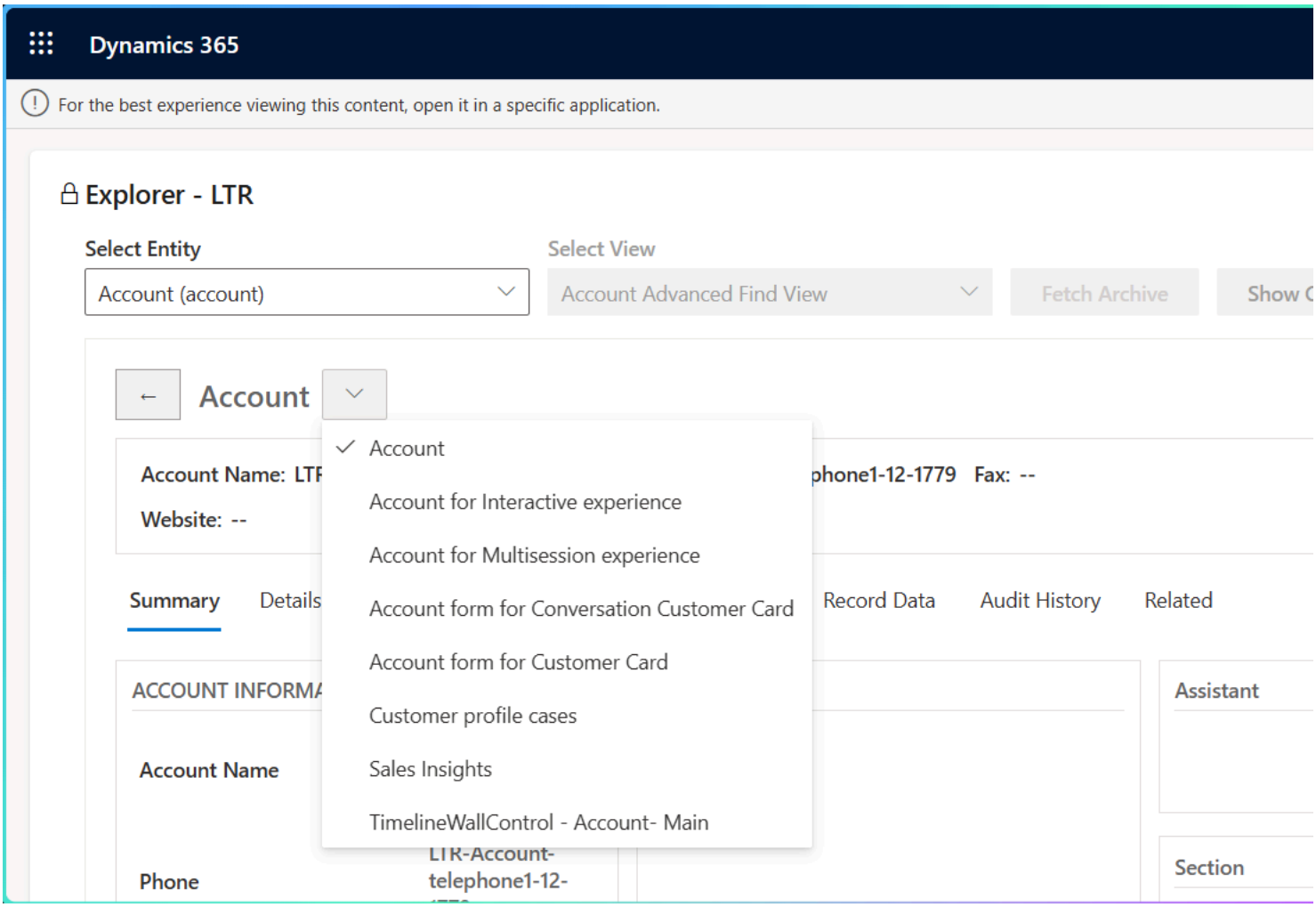
Step 6 - Open a row into detail context

Select a grid row to open detail section and tabs.



Step 7 - Use form switcher

Open the detail form dropdown and choose alternate form layouts when available.



Step 8 - Inspect Record Data tab

Review key-value field output.

Summary | Details | Scheduling | Files | Assets and Locations | **Record Data** | Audit History | Related

Field	Value
_createdby_value	# ltrdisplay
_modifiedby_value	Praveen Thonda
_modifiedonbehalfby_value	Praveen Thonda
_ownerid_value	# ltrdisplay
_owningbusinessunit_value	orgd90897e4
_owninguser_value	d94eef6c-e81c-f111-88b3-7ced8d1dc79f
Account Name	LTR-Account-name-12-6398
accountclassificationcode	Default Value
accountid	aa75508b-d81f-f111-88b3-7c1e5267f8c3
accountratingcode	Default Value
address1_addressid	a8741866-c89f-483c-a455-a82c6d05d5be
address2_addressid	4951a185-2655-443a-98f0-e0b8c4105413

Step 9 - Inspect Audit History tab

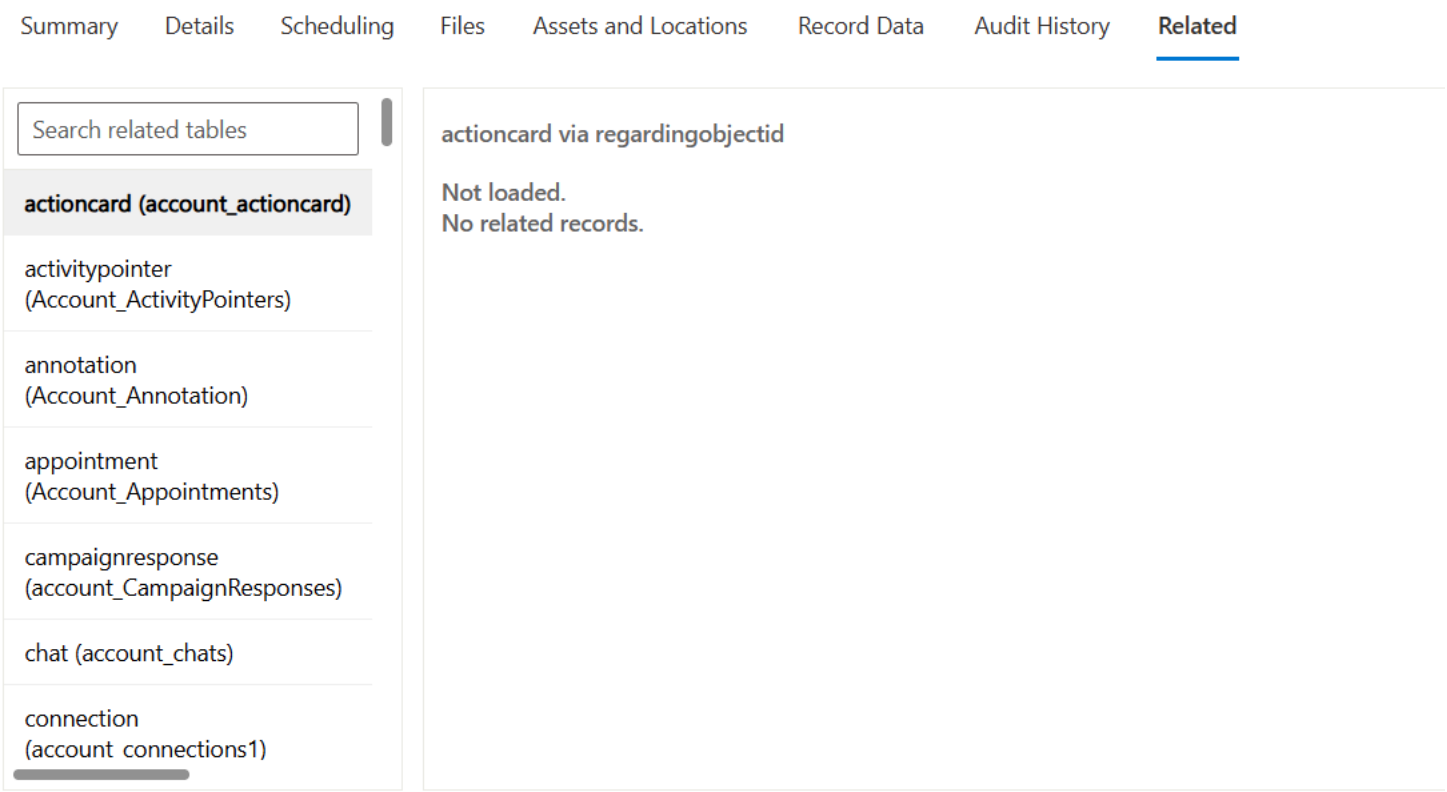
Review changed by, changed on, operation, and old/new values.

Summary | Details | Scheduling | Files | Assets and Locations | Record Data | **Audit History** | Related

Changed On	Changed By	Operation	Action	Attribute	Old Value	New Value
3/14/2026, 2:04:03 PM	# ltrdisplay	Create	Create	accountclassificationcode	--	1
				accountratingcode	--	1
				address2_addresstypecode	--	1
				address2_freighttermscode	--	1
				address2_shippingmethodcode	--	1
				businesstypecode	--	1
				creditonhold	--	False
				customersizecode	--	1
				donotbulkemail	--	False
				donotbulkpostalmail	--	False
				donotemail	--	False

Step 10 - Use Related tab

Select relationship and click Load to fetch related rows lazily.



6. Fork and Customize the Repository

6.1 Fork and Clone

1. Fork the repository in GitHub.
2. Clone your fork locally.
3. Create a feature branch for your changes.

6.2 Local Build Setup

From repository root:

```
npm install
npm run build
```

6.3 Typical Customization Areas

Most teams customize:

- App-level behavior and UX flow: LTRDisplayControl/components/App.tsx
- Grid columns and filter behavior: LTRDisplayControl/components/DynamicGrid.tsx
- Detail tabs and rendering: LTRDisplayControl/components/DynamicForm.tsx
- Dataverse query strategy: LTRDisplayControl/services/LtrService.ts
- Styling and branding: LTRDisplayControl/css/LTRDisplayControl.css

6.4 Push Changes to Dataverse (development loop)

Use your existing PAC workflow in the target environment.

Typical sequence:

```
npm run build
pac pcf push --publisher-prefix ltr --incremental
pac solution publish
```

6.5 Export and Repackage

After validation in environment:

- Export managed and unmanaged solution zips
- Update solution/LTRDisplay_managed_latest.zip and solution/LTRDisplay_unmanaged_latest.zip
- If needed, unpack for review under exports/unpacked_managed and exports/unpacked_unmanaged

6.6 Recommended Contribution Workflow

- Keep changes scoped by feature branch
- Run build before each push
- Capture screenshots for changed UX behavior
- Update docs in docs folder together with code
- Submit PR with a short validation checklist and test evidence

7. Operational Notes and Best Practices

- Use managed package for consumer installation.
- Keep unmanaged package for internal customization scenarios.
- Treat retained fetches as expensive and prefer cache replay when possible.
- Keep related loading on-demand to control query volume.
- Preserve role-based visibility for admin-focused forms.

8. Conclusion

LTRDisplay provides a practical archive exploration interface for Dataverse model-driven apps with strong focus on usability, cache efficiency, and operational control. By combining managed distribution, clear import validation, and a straightforward customization model, teams can adopt it quickly and evolve it safely for enterprise needs.